Java Stream +

Stream

```
List<User> users = ...;
users.stream()
```

```
.filter()       //
.map()          //
.flatMap()      //                List
.sorted()       //
.distinct()     //
.limit(n)       //
.skip(n)        //
```

```
users.stream()
    .filter(u -> u.getAge() > 18)
    .map(User::getName)
```

```
.collect(Collectors.toList())
```

```
Collectors.toList()
Collectors.toSet()
Collectors.toMap(k, v)
Collectors.joining(",")
Collectors.counting()
Collectors.averagingInt(...)
Collectors.summarizingInt(...)
```

---

groupingBy +

```
Collectors.groupingBy(    ,      )
```

1

```java
Map<String, List<String>> map = users.stream()
    .collect(Collectors.groupingBy(
        User::getCity,
        Collectors.mapping(User::getName, Collectors.toList())
    ));
```

2

```java
Map<String, Long> map = users.stream()
    .collect(Collectors.groupingBy(
        User::getCity,
        Collectors.counting()
    ));
```

3

```java
Map<String, Optional<User>> map = users.stream()
    .collect(Collectors.groupingBy(
        User::getCity,
        Collectors.maxBy(Comparator.comparing(User::getAge))
    ));
```

```java
Map<String, Map<String, List<User>>> result = users.stream()
    .collect(Collectors.groupingBy(
        User::getCountry,
        Collectors.groupingBy(User::getCity)
    ));
```

---

partitioningBy + counting

```java
Map<Boolean, Long> result = users.stream()
    .collect(Collectors.partitioningBy(
        u -> u.getAge() > 18,
        Collectors.counting()
    ));
```

toMap + merge function

```
Map<String, String> map = users.stream()
    .collect(Collectors.toMap(
        User::getCity,
        User::getName,
        (a, b) -> a + "/" + b
    ));
```

collectingAndThen

```
List<String> names = users.stream()
    .map(User::getName)
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        Collections::unmodifiableList
    ));
```

→ →

stream → map/filter/sorted → collect(Collectors.xxx)

groupingBy mapping/counting/maxBy/averaging

toMap partitioningBy

= Java